

Operators & expressions

1. What is the primary role of operators in C programming?

- a. To allocate memory dynamically
- b. To perform operations on data**
- c. To manage program flow
- d. To handle input and output

Explanation: Operators in C are used to perform operations on data.

2. What is operator precedence in C?

- a. The order in which operators are evaluated**
- b. The type of data an operator uses
- c. The size of an operator
- d. The number of operands and operator works with

Explanation: Operator precedence determines the order in which operators are evaluated in an expression.

3. What is the primary use of the assignment operator (=)?

- a. To compare two values
- b. To assign a value to a variable**
- c. To perform arithmetic operations
- d. To find the size of a variable

Explanation: The assignment operator = is used to assign a value to a variable.

4. What is an example of a unary operator in C?

- a. +, -
- b. ++, --**
- c. &&, ||
- d. *, /

Explanation: Unary operators like ++ and -- operate on a single operand to modify its value.

5. How is type conversion useful in C programming?

- a. It allows manipulation of strings
- b. It enables one data type to be converted to another**
- c. It helps in debugging errors
- d. It allocates memory for variables

Explanation: Type conversion allows one data type to be converted to another for compatibility or precision.

6. What is implicit type conversion in C?

- a. A conversion specified by the programmer
- b. An error-checking mechanism
- c. Automatic conversion by the compiler**
- d. A method to compare values

Explanation: Implicit type conversion is automatically handled by the compiler during expression evaluation.

7. What happens during explicit type casting in C?

- a. The compiler automatically converts data types
- b. The user manually specifies the conversion**
- c. The variable value is ignored
- d. The size of the variable increases

Explanation: Explicit type casting involves manually specifying the type to convert a variable or value into.

8. What is the syntax for type casting in C?

- a. datatype(variable)
- b. (type)value**
- c. convert(value, type)
- d. datatype->value

Explanation: The syntax for type casting in C is (type)value.

9. What is the purpose of library functions in C?

- a. To simplify coding by providing predefined functions**
- b. To allocate memory dynamically
- c. To improve the compiler's speed
- d. To compare different algorithms

Explanation: Library functions simplify coding by providing predefined functionalities like I/O, math, and string manipulation.

10. Which header file is used for mathematical functions like sqrt()?

- a. <string.h>
- b. <stdlib.h>
- c. <math.h>**
- d. <stdio.h>

Explanation: The <math.h> header file contains functions like sqrt(), pow(), and abs().

11. What is the result of the modulo operator (%)?

- a. The product of two numbers
- b. The quotient of two numbers
- c. The remainder after division**
- d. The absolute difference between numbers

Explanation: The % operator returns the remainder after division of two numbers.

12. Which operator is used to check equality between two values?

- a. !=
- b. ==**
- c. <=
- d. =

Explanation: The == operator is used to check if two values are equal.

13. What does the logical AND operator (&&) do in C?

- a. Returns true if at least one condition is true
- b. Returns false if both conditions are false
- c. Returns true only if both conditions are true**
- d. Negates a condition

Explanation: The logical AND operator returns true only if both conditions are true.

14. How is the increment operator (++) used in C?

- a. To decrease a value by one
- b. To multiply a value by two
- c. To increase a value by one**
- d. To assign a value to a variable

Explanation: The ++ operator increases the value of a variable by one.

15. Which of these is a relational operator in C?

- a. *
- b. ==**
- c. &&
- d. ++

Explanation: The == operator is a relational operator used to compare two values.

16. Which operator is evaluated first in the expression $10 + 3 * 2 - 4 / 2$?

- a. +
- b. *
- c. -
- d. /

Explanation:

According to operator precedence in C, multiplication (*) and division (/) are evaluated before addition (+) and subtraction (-). Between $3 * 2$ and $4 / 2$, the multiplication (*) is evaluated first because it appears earlier when scanned from left to right.

17. If parentheses are added like this: $(10 + 3) * (2 - 4) / 2$, what will the result be?

- a. 0
- b. -6
- c. -13
- d. 13

Explanation:

Evaluate step by step:

1. Parentheses first:
 - $(10 + 3) = 13$
 - $(2 - 4) = -2$
2. Multiplication:
 - $13 * -2 = -26$
3. Division:
 - $-26 / 2 = -13$

However, in integer division, $-26 / 2$ results in -13

18. What is the result of the following code snippet?

```
int a = 5;  
float b = 2.0;  
float result = a / b;
```

- a. 2
- b. 2.0
- c. 2.5
- d. Compilation error

Explanation: The `a` is implicitly converted to `float` before the division, resulting in $5.0 / 2.0 = 2.5$.

19. What happens if you assign a floating-point value to an integer variable?

- a. The fractional part is truncated.
- b. The value is rounded.
- c. A compilation error occurs.
- d. The value is converted to a float.

Explanation: When assigning a float to an integer, the fractional part is dropped, leaving only the integer part. For example, assigning `3.9` to an integer results in `3`.

20. What happens when you cast a large `long` value to a smaller `short` type in C?

- a. The value will be truncated or cause data loss.
- b. The value will be automatically adjusted.
- c. The result will cause a compilation error.
- d. The value will be rounded.

Explanation:

Casting a large `long` value to a smaller `short` type may cause data loss or truncation because the range of `short` is smaller than that of `long`.

